VIET NAM NATIONAL UNIVERSITY HO CHI MINH

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Shortest Path Algorithms

Seminar

Course: Data Structures - CS163

Student list:

Nguyễn Anh Kiệt (25125074)

Trần Như Khải (25125045)

Nguyễn Đức Huy (25125014)

Nguyễn Đình Minh Huy (25125083)

Group 05 - Class 25A01

19th April 2026

Contents

1 Introduction & History **2**

1.1 Problem Introduction . . . 2

1.2 The Evolution of Shortest Path Algorithms . . . 2

1.3 Shortest Paths in Today's World . . . 3

2 Main Data Structures & Algorithms **3**

3 Single Source Shortest Path (SSSP) **3**

3.1 Shortest Path in DAGs . . . 3

3.1.1 Topological Order . . . 4

3.1.2 DAG Shortest Path Processing . . . 5

3.1.3 Step-by-Step Demonstration: DAGs . . . 5

3.2 Dijkstra's Algorithm . . . 8

3.2.1 Dijkstra Optimizing . . . 9

3.2.2 Step-by-Step Demonstration: Dijkstra's Algorithm . . . 10

3.3 Shortest Path in Bellman-Ford . . . 12

3.3.1 How Bellman-Ford works: Global Relaxation . . . 12

3.3.2 Bellman-Ford Superpower: Negative Cycle Detection . . . 13

3.3.3 Step-by-Step Demonstration: Bellman-Ford Algorithm . . . 15

3.4 SSSP Algorithm: How to track the path? . . . 17

4 All Pairs Shortest Path (APSP) **18**

4.1 Floyd-Warshall Algorithm . . . 18

4.1.1 The Intermediate Node . . . 18

4.1.2 How to track the path? . . . 19

4.1.3 Step-by-Step Demonstration: Floyd-Warshall Algorithm . . . 20

4.2 A* Search Algorithm . . . 23

4.2.1 Step-by-Step Demonstration: A* . . . 24

4.3 Johnson's Algorithm . . . 24

4.3.1 How Johnson's Algorithm works . . . 25

4.3.2 Step-by-Step Demonstration: Johnson's Algorithm . . . 26

5 Multi-Source Shortest Path (MSSP) 27

5.1 Super Source Simulation . . . 27

5.1.1 Step-by-Step Demonstration: MSSP . . . 29

6 Implementation Notes & Complexity 29

6.1 Implementation Notes . . . 29

6.2 Complexity Notes in Practice . . . 30

6.3 Engineering Trade-offs . . . 30

7 Applications 30

8 Quiz 30

9 Programming Assignments 30

10 Source Codes & Instructions 30

10.1 Dependencies . . . 31

10.2 What Is Included . . . 31

10.3 Input Graph Format . . . 31

10.4 Compile Instructions (Maximum Optimization) . . . 31

10.4.1 macOS (clang++) . . . 32

10.4.2 Windows (MSYS2/MinGW-w64 g++) . . . 32

10.4.3 Windows (MSVC cl) . . . 33

10.5 Run Workflow (After Compilation) . . . 33

10.6 Generate the Benchmark Datasets . . . 35

10.7 Generate the DAG Benchmark Dataset . . . 35

11 Datasets 35

11.1 Dataset Suite . . . 35

11.2 Dataset Files . . . 35

11.3 Header Format . . . 36

11.4 Benchmark Interpretation . . . 36

12 Comparison 36

References **37**

Group Information

This report is conducted by Group 05. Below is the list of group members:

Student ID	Full Name
25125074	Nguyễn Anh Kiệt
25125045	Trần Như Khải
25125014	Nguyễn Đức Huy
25125083	Nguyễn Đình Minh Huy

1 Introduction & History

1.1 Problem Introduction

In our daily lives, we constantly face the problem of finding the optimal route. Whether it is a delivery driver planning their route, or a commuter asking, "Which way is the fastest?", the underlying challenge remains the same. The question then arises: *How do computers navigate thousands of turns and intersections to find the absolute best path in mere milliseconds?* To answer this, we must look into Graph Theory and the Shortest Path algorithms that power modern navigation systems.

1.2 The Evolution of Shortest Path Algorithms

The quest to find optimal paths is not a modern computing problem; its roots trace back centuries, evolving through significant mathematical and computational breakthroughs:

- **1736 - The Birth of Graph Theory:** The foundation of all routing algorithms was laid by Leonhard Euler when he famously solved the Seven Bridges of Königsberg problem. By abstracting the landmasses as nodes and bridges as edges, he proved that no path could cross all seven bridges exactly once, officially giving birth to Graph Theory.
- **1956–1958 - Bellman-Ford Algorithm:** Richard Bellman and Lester Ford published an algorithm capable of finding the shortest paths from a single source vertex to all other vertices. Crucially, this algorithm successfully finds shortest paths even with negative edge weights, making it highly versatile for complex models.
- **1959 - Dijkstra's Algorithm:** Edsger W. Dijkstra published his legendary algorithm. It became the gold standard for finding the absolute shortest path from point A to point B in graphs with non-negative edge weights.
- **1962 - Floyd-Warshall Algorithm:** Robert Floyd and Stephen Warshall introduced a dynamic programming approach to compute the shortest paths between *all pairs* of nodes simultaneously within a graph.
- **1968 - A* Search Algorithm:** As systems grew more complex, pure graph traversal became computationally heavy. The A* algorithm was created as a faster shortest path finder, utilizing

heuristics to actively guide the AI search direction rather than exploring in all directions equally.

1.3 Shortest Paths in Today's World

While the foundational algorithms of the mid-20th century are brilliant, representing the real world today—such as the infrastructure behind Google Maps—requires graphs with billions of nodes. Running traditional shortest-path algorithms across an entire country's road network would be fundamentally too slow for real-time user queries.

To overcome this, researchers introduced advanced routing techniques like **Contraction Hierarchies (2008)** and **Hub Labeling**. By extensively pre-computing the map data, modern servers can calculate the shortest path across enormous geographical distances in mere milliseconds.

2 Main Data Structures & Algorithms

In a weighted graph, the objective of a shortest path algorithm is to identify a sequence of edges connecting vertices such that the sum of the edge weights is minimized. The problem is generally bifurcated into two primary categories:

- **Single Source Shortest Path (SSSP):** Determines the minimum cost to travel from a single starting node (the source) to all other reachable nodes in the graph.
- **All Pairs Shortest Path (APSP):** Computes the shortest paths between every possible pair of vertices simultaneously.

3 Single Source Shortest Path (SSSP)

The Single Source Shortest Path (SSSP) problem asks a fundamental question: *What is the shortest path from one starting node to all other nodes?*

3.1 Shortest Path in DAGs

Historical Context: While shortest path traversal on Directed Acyclic Graphs represents a fundamental application of graph theory rather than a single named algorithm, its core mechanism—Topological

Sorting—was mathematically formalized by Arthur B. Kahn in 1962. The sequential evaluation method used to find shortest paths has direct roots in the PERT (Program Evaluation and Review Technique) charts developed by the United States Navy in the 1950s for project scheduling.

A **Directed Acyclic Graph (DAG)** is a directed graph which forms no cycles or loops. The SSSP problem can be solved efficiently on a DAG due to the fact that the nodes can be ordered in a topological ordering via topsort and processed sequentially.

3.1.1 Topological Order

A *linear ordering of its vertices such that for every directed edge from vertex u to vertex v ($u \to v$), vertex u comes before vertex v in the ordering.*

Topological sorting can be implemented in two standard ways: a DFS-based approach or a BFS-based approach (Kahn's algorithm). Kahn's algorithm is especially useful when we want built-in DAG validation, because it can detect non-DAG inputs when not all vertices can be processed. In this implementation, we use the DFS-based method and assume the input graph is already a DAG.

Topological Sort Pseudocode:

Algorithm 1 Topological Sort

```
function TOPOLOGICALSORT(graph)
    visited ← [false, false, ..., false]                ▷ size V
    order ← empty stack
    for i ← 0 to V − 1 do
        if not visited[i] then
            DFS(graph, i, visited, order)
        end if
    end for
    return order
end function
function DFS(graph, index, visited, order)
    visited[index] ← true
    for edge in graph[index] do
        if not visited[edge.to] then
            DFS(graph, edge.to, visited, order)
        end if
    end for
    order.push(index)
end function
```

3.1.2 DAG Shortest Path Processing

1. Select the first coming vertex from the topological ordering.
2. Explore all its neighbours.
3. If finding a shorter path, relax the edge.
4. Reprocess until finishing.

3.1.3 Step-by-Step Demonstration: DAGs

Phase 1: Graph Structure and Initialization

1. **Adjacency List Setup:** The visualized graph consists of four nodes (0, 1, 2, 3) with directed, weighted edges:
   - Node 0 connects to Node 1 (weight 1) and Node 2 (weight 4).
   - Node 1 connects to Node 2 (weight 2).
   - Node 2 connects to Node 3 (weight 3).
   - Node 3 has no outgoing edges (terminal node).
2. **DFS Execution and Stack:** The visualizer performs a Depth-First Search (DFS) starting from Node 0 to find the topological order. The discovery order pushes nodes sequentially into the stack: `push 0` $\rightarrow$ `push 1` $\rightarrow$ `push 2` $\rightarrow$ `push 3`. Consequently, the processing topological order is `[0, 1, 2, 3]`.
3. **Distance Array (`dist`):** The `dist` table at the bottom is initialized. The starting node (Node 0) is set to 0, while all other nodes are set to ∞ before the search begins.

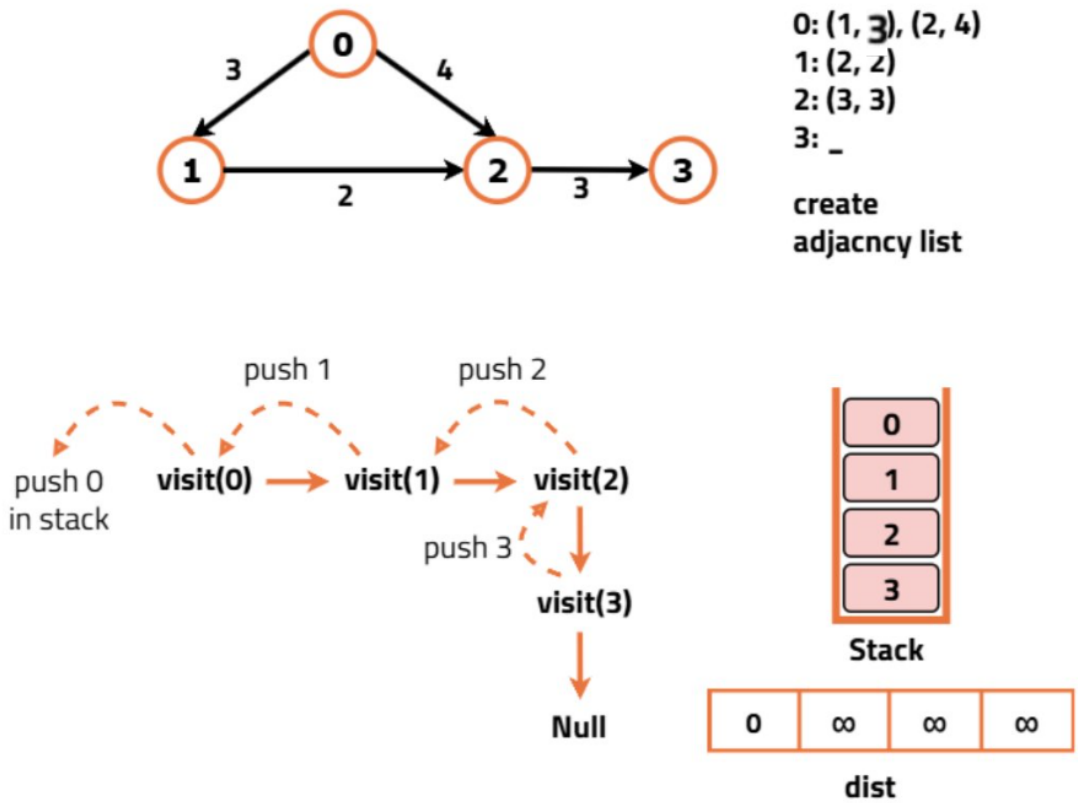

Figure 1: Phase 1 - DFS topological sort pushing nodes into the stack, and the initialization of the `dist` array.

Phase 2: Sequential Processing and Edge Relaxation

4. **Step 1 (Process Node 0):** The algorithm pops Node 0 from the stack. It highlights and relaxes the outgoing edges to its neighbors:
   - `dist[1] = min(∞, 0 + 1) = 1`
   - `dist[2] = min(∞, 0 + 4) = 4`
5. **Step 2 (Process Node 1):** Node 1 is popped. The algorithm evaluates the edge pointing to Node 2:
   - `dist[2] = min(4, 1 + 2) = 3` (The distance is successfully updated dynamically on the screen because a shorter path through Node 1 was found).
6. **Step 3 (Process Node 2):** Node 2 is popped. The edge pointing to Node 3 is evaluated:

- `dist[3] = min(∞, 3 + 3) = 6`

7. **Step 4 (Process Node 3):** The terminal Node 3 is popped. Since there are no outgoing edges, no changes are made.

8. **Final Result:** The algorithm terminates. The final shortest path distances from Node 0 to all other nodes are cleanly displayed as `[0, 1, 3, 6]`.

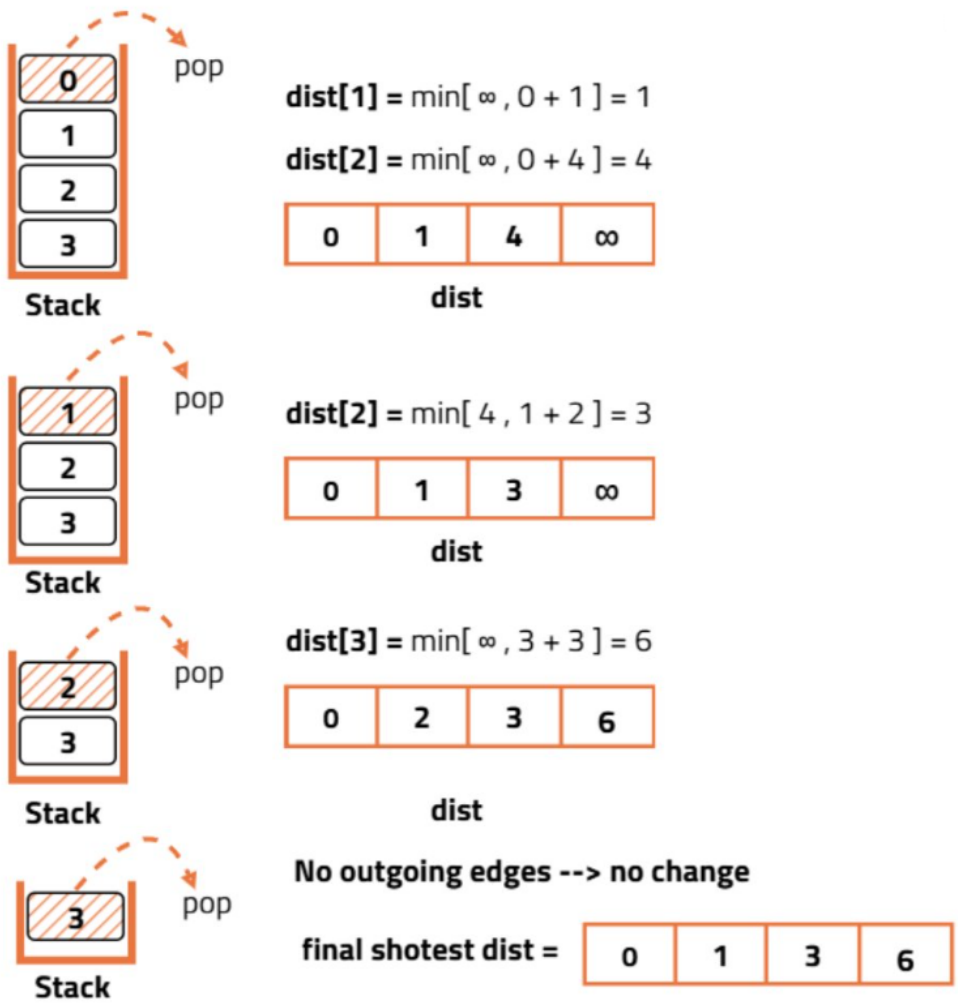

Figure 2: Phase 2 - Sequential traversal follows the topological order, dynamically relaxing edges and updating the shortest distances.

DAG Shortest Path Pseudocode:

Algorithm 2 DAG Shortest Path

```
function dagShortestPath(graph, start)
    distances ← [∞, ∞, ..., ∞]                    ▷ size V
    distances[start] ← 0
    topoOrder ← topologicalSort(graph)
    while topoOrder is not empty do
        index ← topoOrder.pop()
        if distances[index] == ∞ then
            continue
        end if
        for edge in graph[index] do
            newDist ← distances[index] + edge.cost
            if newDist < distances[edge.to] then
                distances[edge.to] ← newDist
            end if
        end for
    end while
    return distances
end function
```

Complexity Analysis:

- **Time Complexity: O(V + E)**. We are traversing every vertex and edge.
- **Space Complexity: O(V)**. We are storing the topological ordering and the distances array.

3.2 Dijkstra's Algorithm

Historical Context: Conceived by the Dutch computer scientist Edsger W. Dijkstra in 1956 (and formally published in 1959 in *A note on two problems in connexion with graphs*), this algorithm was famously designed in just twenty minutes while Dijkstra was drinking coffee at a café in Amsterdam. He initially created it to find the shortest path between the cities of Rotterdam and Groningen to demonstrate the capabilities of the new ARMAC computer.

Given a graph of non-negative weight edges. We can greedily assume that adding edges can only increase the total path cost.

Therefore, we come up with the **Greedy strategy**: continually select the unvisited node with the smallest known distance using a min-heap assuming it cannot be improved anymore, and then relaxes its outgoing edges.

Shortest Path Pseudocode:

Algorithm 3 Dijkstra's Algorithm

```
function DIJKSTRA(graph, start)
    visited ← [false, false, ..., false]                ▷ size V
    distances ← [∞, ∞, ..., ∞]                          ▷ size V
    distances[start] ← 0
    pq ← empty priority queue (min-heap)
    pq.insert({0, start})
    while pq is not empty do
        index, minValue ← pq.poll()
        visited[index] ← true
        if distances[index] < minValue then
            continue
        end if
        for edge in graph[index] do
            if visited[edge.to] then
                continue
            end if
            newDist ← distances[index] + edge.cost
            if newDist < distances[edge.to] then
                distances[edge.to] ← newDist
                pq.insert({edge.to, newDist})
            end if
        end for
    end while
    return distances
end function
```

Complexity Analysis:

- **Time Complexity:** $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$. We extract exactly V vertices. Each extraction from a heap takes $O(\log V)$. We evaluate at most E edges. If we relax the edge, we push new value into the heap takes $O(\log V)$.
- **Space Complexity:** $\mathbf{O}(\mathbf{V}+\mathbf{E})$. We are storing the distances array for every vertex and the priority queue for each relaxed edge.

3.2.1 Dijkstra Optimizing

- **Lazy Dijkstra:** uses a standard Binary Heap. Time complexity: $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$.
- **Eager Dijkstra:** uses an indexed priority queue to instantly locate the vertex and update the value dynamically. Time complexity: $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$.

- **D-ary Heap Dijkstra (dense graph optimizer):** uses a Heap giving each parent D children, cheaper when updating but more expensive when polling.
  Time complexity: $\mathbf{O}(\mathbf{V}\log_{\mathbf{D}}\mathbf{V} + \mathbf{E}\log_{\mathbf{D}}\mathbf{V})$.

- **Fibonacci Heap Dijkstra:** Replaces the standard tree-based heap with a highly complex collection of trees called a Fibonacci Heap which achieved updating time an amortized $O(1)$.
  Time Complexity: $\mathbf{O}(\mathbf{V}\log\mathbf{V} + \mathbf{E})$.

3.2.2 Step-by-Step Demonstration: Dijkstra's Algorithm

This section describes the step-by-step execution of Dijkstra's algorithm as captured in our visualizer, illustrating the movement between the Open and Closed sets and the relaxation of edges.

Phase 1: Initialization of the Source Node

1. **Source Selection:** We select **Node A** as the starting point. The distance from A to itself is naturally 0, thus the `Distance` column for A in the Open Set table is updated to 0.

2. **Initial State:** All other nodes (B, C, D, E, F) are initially undiscovered and assigned a distance of infinity (∞).

3. **Open Set Management:** At this stage, the `Open Set` contains all vertices. The system identifies the node with the minimum distance to begin processing, which is Node A (highlighted in blue).

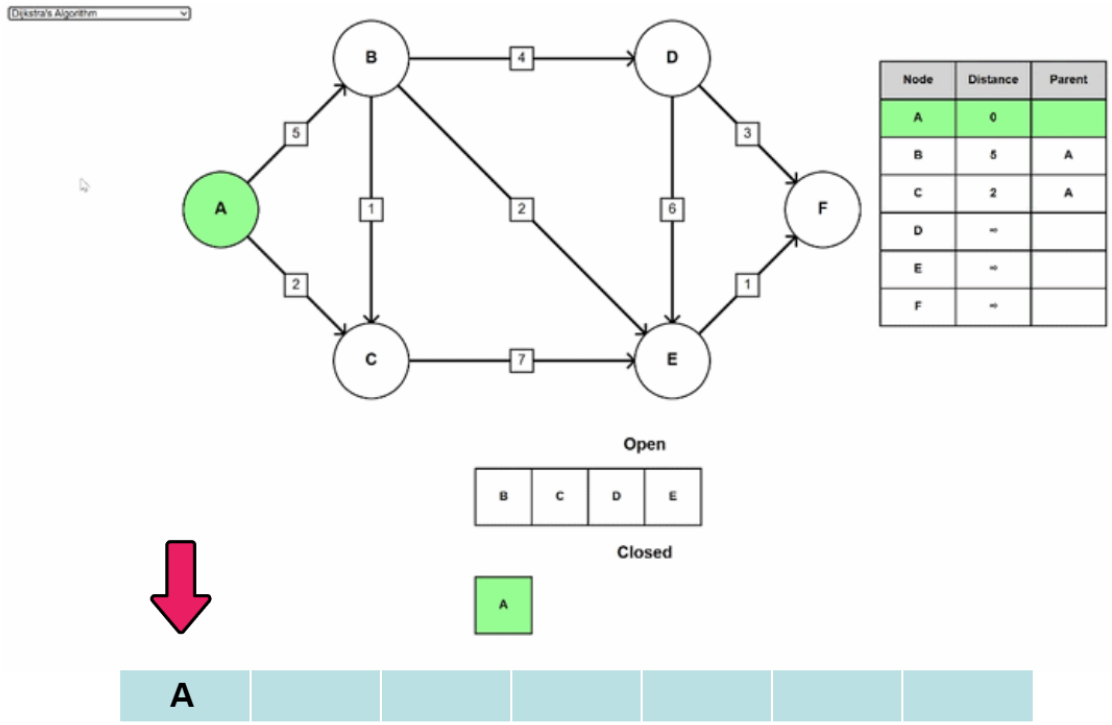

Figure 3: Dijkstra's Algorithm - Phase 1: Node A is selected as the current node, and its neighbors (B and C) are being evaluated.

Phase 2: Edge Relaxation (Updating Neighbors)

4. **Scanning Neighbors:** From the current Node A, the algorithm "looks" at its immediate neighbors. As indicated by the orange arrows in the visualizer, A has directed edges to B and C.

5. **Relaxing Edge A → B:** The edge weight is 5. The algorithm calculates the potential distance: $0 + 5 = 5$. Since $5 < \infty$, the distance for **Node B** is updated to 5, and Node A is recorded as its parent (as shown in the yellow highlighted cell in the table).

6. **Relaxing Edge A → C:** Similarly, the edge weight is 2. The calculation $0 + 2 = 2$ is performed. Since $2 < \infty$, the distance for **Node C** is updated to 2, with Node A set as its parent.

Phase 3: Finalizing Node and Selecting the Next Best Path

7. **Closing the Node:** Once all neighbors of A are checked, Node A is completed. It is removed

from the `Open Set` and moved to the `Closed Set`. We are now guaranteed that the shortest path to A has been found.

8. **Next Iteration:** The algorithm returns to the `Open Set` to find the node with the current minimum distance. Comparing B (Dist: 5) and C (Dist: 2), Node C is clearly the smallest.

9. **Expansion:** Node C will be selected as the next "Current Node" in the following step. The process will repeat: checking C's neighbor (Node E), calculating cumulative distances, and updating the tables until the target is reached or all nodes are closed.

3.3 Shortest Path in Bellman-Ford

Historical Context: This algorithm primarily bears the names of Richard Bellman (who published it in 1958 in *On a routing problem*) and Lester R. Ford Jr. (who developed the foundational flow formulations in 1956). However, the algorithm was independently published by Edward F. Moore in 1959 and Alfonso Shimbel in 1955. As a pioneering application of Bellman's principle of optimality, it introduced dynamic programming to the realm of graph routing.

When a graph contains negative weight edges, Dijkstra's greedy logic break down. What should we do?

Bellman-Ford is a Single Source Shortest Path (SSSP) algorithm designed specifically to handle graphs with negative weights. Unlike Dijkstra (Greedy), Bellman-Ford uses **Dynamic Programming**. It calculates shortest paths in a bottom-up manner by gradually allowing paths to use more and more edges.

3.3.1 How Bellman-Ford works: Global Relaxation

- **Step 1: Initialization.** Set the distance to the source node to 0 and all other nodes to ∞.

- **Step 2: Edge Relaxation.** Iterate through every single edge (u, v) with weight w in the graph. If a new shorter path is found, then relax that edge.

- **Step 3: Repeat.** Perform this full-graph relaxation exactly $\mathbf{V - 1}$ **times**.

Why V - 1 times? We expect after one loop the path is extended by at least one edge. The shortest path in a graph with V vertices without cycles, contains at least $V - 1$ edges.

3.3.2 Bellman-Ford Superpower: Negative Cycle Detection

Negative cycle: *A loop in the graph where the sum of the edge weights is less than zero. If one exists, you can loop infinitely to achieve a distance of $-\infty$. No shortest path can be defined.*

After running $V - 1$ iterations, run the relaxation one more time. If any edge still can be relaxed, that edge is in a negative cycle. All vertices can be routed from the negative vertices have $-\infty$ distance.

Bellman-Ford Pseudocode:

Algorithm 4 Bellman-Ford Algorithm

```
function BELLMANFORD(graph, start)
    dist ← [∞, ∞, ..., ∞]                                   ▷ size V
    dist[start] ← 0
    for i ← 1 to V − 1 do
        for edge in graph.edges do
            if dist[edge.from] + edge.cost < dist[edge.to] then
                dist[edge.to] ← dist[edge.from] + edge.cost
            end if
        end for
    end for
    negVertices ← empty queue
    for edge in graph.edges do
        if dist[edge.from] + edge.cost < dist[edge.to] then
            dist[edge.to] ← −∞
            negVertices.enqueue(edge.to)
        end if
    end for
    PROCESSNEGATIVECYCLE(graph, negVertices, dist)
    return dist
end function
function PROCESSNEGATIVECYCLE(graph, negVertices, dist)
    visited ← [false, false, ..., false]                    ▷ size V
    while negVertices is not empty do
        vertex ← negVertices.dequeue()
        if visited[vertex] then
            continue
        end if
        DFS(graph, vertex, negVertices, dist, visited)
    end while
end function
function DFS(graph, vertex, negVertices, dist, visited)
    visited[vertex] ← true
    for edge in graph[vertex] do
        if visited[edge.to] then
            continue
        end if
        dist[edge.to] ← −∞
        negVertices.enqueue(edge.to)
        DFS(graph, edge.to, negVertices, dist, visited)
    end for
end function
```

Complexity Analysis:

- **Time Complexity: $\mathbf{O(E \cdot V)}$**. We iterate every edges $V - 1$ times and one more loop for negative cycle detection.
- **Space Complexity: $\mathbf{O(V)}$**. We are storing the distances array for every vertex and the priority queue for each relaxed edge.

Data Insight (Placeholder):

on the dataset.

3.3.3 Step-by-Step Demonstration: Bellman-Ford Algorithm

This section details the internal mechanics of the Bellman-Ford algorithm during its first iteration, highlighting the relaxation process and the final validation for negative cycles.

Phase 1: Initialization and Starting State

1. **Initial Configuration:** The source vertex **S** is assigned a distance of 0. To account for the worst-case scenario before discovery, all other vertices (A, B, C, D, E) are initialized with a distance of infinity (∞).
2. **Global Relaxation Principle:** Unlike Dijkstra, Bellman-Ford operates by relaxing all E edges in the graph. In our implementation, new values are reused immediately within the same iteration, creating a "Domino Effect" that accelerates convergence.

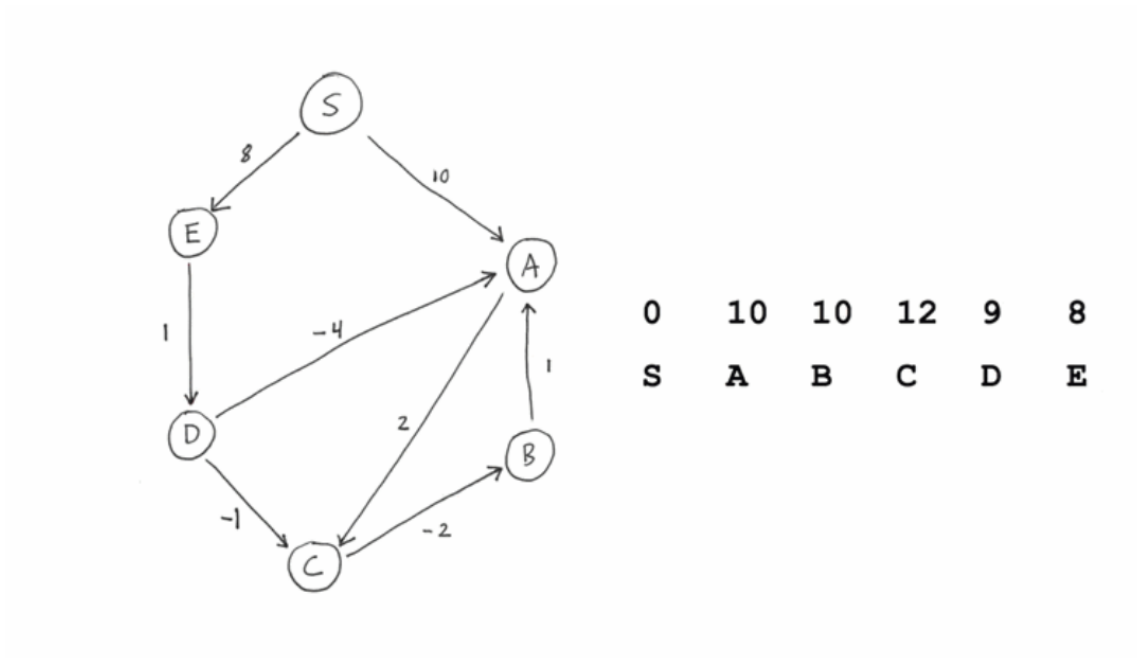

Figure 4: Initial state of the Bellman-Ford visualizer: Node S is initialized to 0, and all other nodes are set to ∞.

Phase 2: The Relaxation Process (Iteration 1) Based on the specific edge list traversal order, the first iteration proceeds as follows:

- **Skipped Edges:** Edges `D` $\rightarrow$ `A`, `D` $\rightarrow$ `C`, and `B` $\rightarrow$ `A` are evaluated first. Since the distance to D and B is still ∞, these updates are ignored.
- **Direct Relaxation from Source:**
  - `S` $\rightarrow$ `A` (weight 10): $0 + 10 = 10$. Distance to A becomes 10.
  - `S` $\rightarrow$ `E` (weight 8): $0 + 8 = 8$. Distance to E becomes 8.
- **Domino Effect (Value Propagation):**
  - `A` $\rightarrow$ `C` (weight 2): Using A's new value (10), $10 + 2 = 12$. Distance to C becomes 12.
  - `C` $\rightarrow$ `B` (weight -2): Using C's new value (12), $12 - 2 = 10$. Distance to B becomes 10.
  - `E` $\rightarrow$ `D` (weight 1): Using E's value (8), $8 + 1 = 9$. Distance to D becomes 9.

Phase 3: Result Analysis and Convergence

3. **Post-Iteration 1 Summary:** After the first pass, the distance array is recorded as: `S:0, A:10, B:10, C:12, D:9, E:8`.

4. **Final Convergence:** For a graph with $V = 6$ nodes, the algorithm will perform up to $V - 1 = 5$ iterations. This ensures that even the most distant nodes or those connected by negative edges (like `D` $\rightarrow$ `A`) are properly optimized.

5. **Negative Cycle Detection:** After 5 iterations, the visualizer performs one final "check" pass. If any edge $u \rightarrow v$ still satisfies $dist[u] + weight < dist[v]$, the system highlights the cycle in red and concludes that a **Negative Weight Cycle** exists.

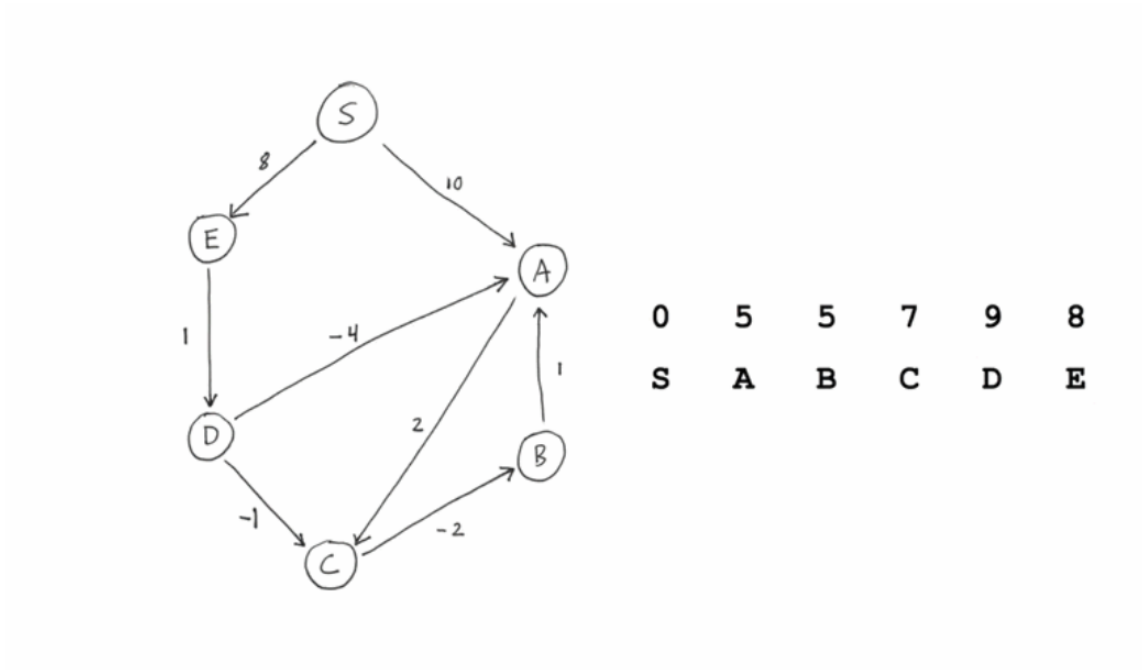

Figure 5: Final state after convergence: The visualizer confirms the shortest paths and validates the absence of negative cycles.

3.4 SSSP Algorithm: How to track the path?

By introducing a `prev` array, where `prev[i]` denotes its previous vertex. We simply update it whenever relaxing the edges:

```
if dist[edge.from] + edge.cost < dist[edge.to] then
    dist[edge.to] ← dist[edge.from] + edge.cost
    prev[edge.to] ← edge.from
end if
```

Then when we need to track the path, we can route from the end to start through the `prev` array then reversing that result.

4 All Pairs Shortest Path (APSP)

What is the shortest path between all pairs of nodes?

4.1 Floyd-Warshall Algorithm

Historical Context: Published independently by Robert W. Floyd (*Algorithm 97: Shortest Path*, 1962) and Stephen Warshall (*A theorem on Boolean matrices*, 1962), this algorithm is an elegant realization of dynamic programming. It fundamentally relies on a theorem Warshall initially developed for computing the transitive closure of a graph, which Floyd ingeniously adapted to compute minimal path weights.

We are no longer looking for paths from a single source. We want to find the shortest path between every pair of vertices (u, v) simultaneously.
Floyd-Warshall is an APSP dynamic programming algorithm which relies on the idea of comparing between direct connection and routing through an intermediate vertex.

4.1.1 The Intermediate Node

Can we find a shorter path from i to j by routing it through an intermediate node k?

- **Initialization:** Create a 2D matrix where `dist[i][j]` is the weight of the direct edge from i to j (or ∞ if no direct edge exists).
- We iterate V loops with index of k, that $\texttt{dist}[i][k] + \texttt{dist}[k][j]$ is the distance when routing through vertex k. If it is better than the current connection, we update $\texttt{dist}[i][j]$. We are processing for every pair (i, j).

Floyd-Warshall Pseudocode:

Algorithm 5 Floyd-Warshall Algorithm

```
function SETUP(graph, dist)
    for i ← 0 to V − 1 do
        dist[i][i] ← 0
    end for
    for edge in graph.edges do
        dist[edge.from][edge.to] ← edge.cost
    end for
end function
function FLOYDWARSHALL(graph)
    dist ← 2D array of size V × V initialized to ∞
    SETUP(graph, dist)
    for k ← 0 to V − 1 do
        for i ← 0 to V − 1 do
            for j ← 0 to V − 1 do
                if dist[i][k] + dist[k][j] < dist[i][j] then
                    dist[i][j] ← dist[i][k] + dist[k][j]
                end if
            end for
        end for
    end for
    return dist
end function
```

Complexity Analysis:

- **Time Complexity: $\mathbf{O(V^3)}$**. We iterate V loops to update for every pair of (i, j).
- **Space Complexity: $\mathbf{O(V^2)}$**. We only use a 2D matrix to store the distance between every pair of vertices.

4.1.2 How to track the path?

By introducing a `next` 2D matrix, where `next[i][j]` denotes the next node from i when routing to j. We initialize `next[i][j] = j` if there is a direct connection or `next[i][j] = -1`. After initializing, in the main loop, we update the `next` matrix as well as the distance:

```
if dist[i][k] + dist[k][j] < dist[i][j] then
    dist[i][j] ← dist[i][k] + dist[k][j]
    next[i][j] ← k
end if
```

Then when we need to track the path, we can route from the start to end through the `next` matrix.

4.1.3 Step-by-Step Demonstration: Floyd-Warshall Algorithm

This section describes the step-by-step execution of the Floyd-Warshall algorithm as captured in our visualizer, demonstrating the dynamic programming matrix updates.

Phase 1: Initialization

1. **Matrix Setup:** The system initializes a 2D distance matrix `dist` of size $V \times V$ (for this 4-node graph, a 4×4 matrix).

2. **Self-Distances:** The main diagonal is set to 0 (`dist[i][i] = 0`).

3. **Direct Edges:** For every direct edge from i to j with weight w, the matrix is updated to `dist[i][j] = w`. For example, `dist[0][1] = 2` and `dist[3][0] = -2`.

4. **No Direct Path:** Pairs of nodes without a direct connecting edge are initialized to infinity (∞). For example, `dist[0][2] = ∞`.

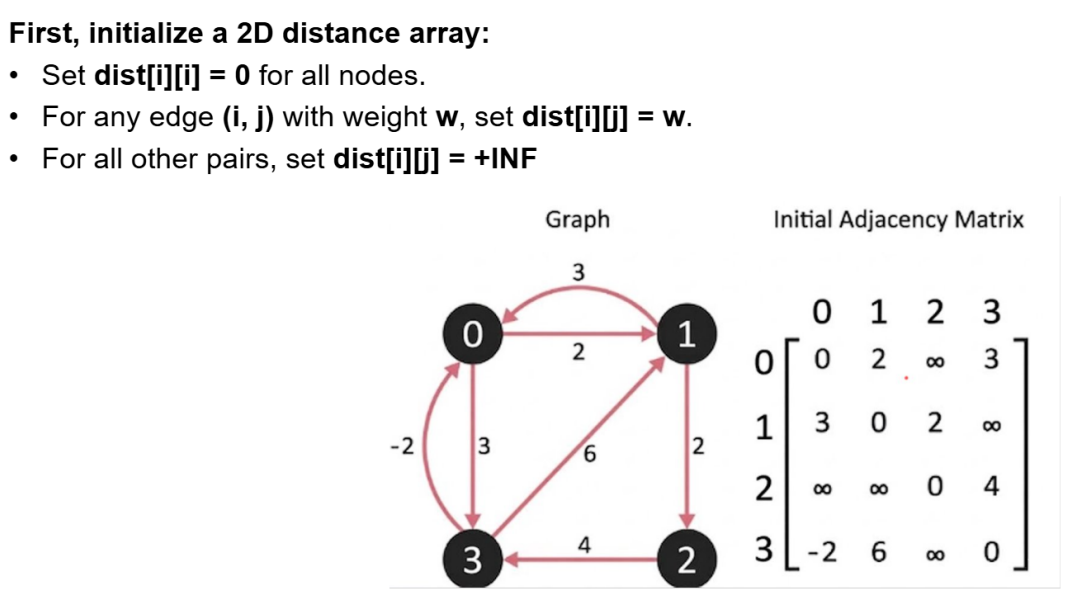

Figure 6: Phase 1 - Initialization of the 4×4 Adjacency Matrix before processing.

Phase 2: Relaxation via Intermediate Node ($k = 0$)

5. **The Outer Loop:** The algorithm selects each node k (from 0 to $V - 1$) to act as an intermediate routing node. At each step k, it checks if routing from i to j through k is shorter using the formula:

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$$

6. **Processing Node 0:** We analyze the first iteration where the intermediate node is **Node 0** ($k = 0$). *(Note: While the UI title says 'Vertex 1' due to 1-based numbering conventions, the mathematical logic is processing index 0).* Row 0 and Column 0 act as fixed references for this loop.

7. **Case 1 - No Update:** The algorithm evaluates the cell `dist[1][1]`. The path routing through Node 0 is calculated as `dist[1][0] + dist[0][1] = 3 + 2 = 5`. Since $5 > 0$ (the current recorded distance), the value is discarded and not updated.

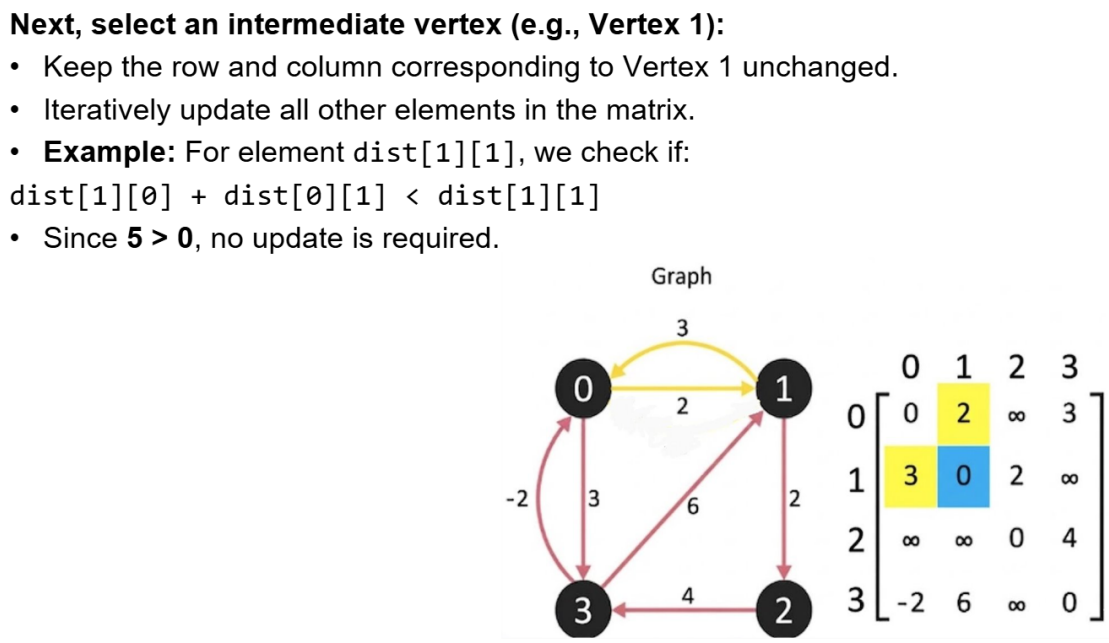

Figure 7: Phase 2a - Evaluating `dist[1][1]` through Node 0. The new distance (5) is greater than the current (0), resulting in no update.

8. **Case 2 - Successful Relaxation:** The algorithm evaluates `dist[3][1]` (path from 3 to 1).

   - Current distance: `dist[3][1] = 6`
   - New distance (via Node 0): `dist[3][0] + dist[0][1] = -2 + 2 = 0`

   Since $0 < 6$, the path through Node 0 is strictly shorter. The system immediately updates `dist[3][1] = 0` (highlighted in blue on the UI).

9. **Further Updates:** Similarly, the cell `dist[1][3]` is updated from ∞ to 6 because `dist[1][0] + dist[0][3] = 3 + 3 = 6`. This concludes the $k = 0$ loop, yielding an optimized matrix.

Next, select an intermediate vertex (e.g., Vertex 1):

- Keep the row and column corresponding to Vertex 1 unchanged.
- Iteratively update all other elements in the matrix.
- **Example:** For element `dist[3][1]`, we check if:

`dist[3][0] + dist[0][1] < dist[3][1]`

- Since **0<6**, We will update this element.

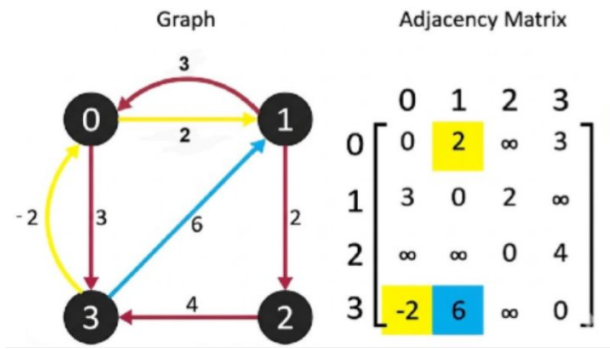

Figure 8: Phase 2b - Successful relaxation of `dist[3][1]`. The cell is highlighted in blue as it updates from 6 to the new minimum of 0.

Phase 3: Completion and Final Result

10. **Subsequent Iterations:** The system repeats this entire process, systematically selecting Node 1, Node 2, and Node 3 as the intermediate pivot ($k = 1, 2, 3$). Each iteration builds upon the matrix output from the previous step.

11. **Final Output:** Once all 4 nodes have served as the intermediate node, the algorithm terminates. The resulting 2D array is the definitive All-Pairs Shortest Path (APSP) matrix, containing the absolute minimum travel cost between any two nodes in the graph.

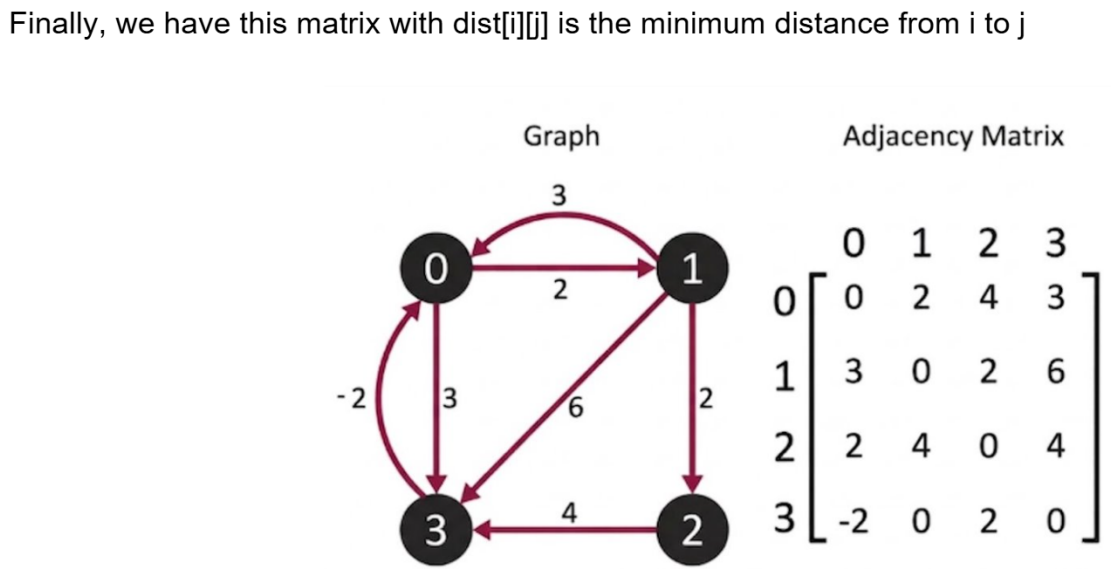

Figure 9: Phase 3 - The final completed matrix containing the optimal shortest paths between all pairs of vertices.

4.2 A* Search Algorithm

Historical Context: Developed in 1968 by Peter Hart, Nils Nilsson, and Bertram Raphael at the Stanford Research Institute, A* was originally designed to help the Shakey robot navigate through rooms containing obstacles.

While Dijkstra's algorithm searches equally in all directions, A* uses a **heuristic** to guide the search direction towards the target, drastically reducing the number of explored nodes. It evaluates nodes by combining two metrics:

$$f(n) = g(n) + h(n)$$

- $g(n)$: The exact cost of the path from the starting node to node n (identical to Dijkstra).
- $h(n)$: The heuristic estimated cost from node n to the goal (e.g., Euclidean or Manhattan distance).
- $f(n)$: The total estimated cost of path through node n.

A* Search Pseudocode:

Algorithm 6 A* Search Algorithm

```
function ASTAR(graph, start, goal)
    gScore ← [∞, ..., ∞]
    gScore[start] ← 0
    pq ← empty priority queue
    pq.insert({0 + H(start, goal), start})                ▷ {fScore, node}
    while pq is not empty do
        current ← pq.poll()
        if current == goal then
            return gScore[current]
        end if
        for edge in graph[current] do
            tentative_gScore ← gScore[current] + edge.cost
            if tentative_gScore < gScore[edge.to] then
                gScore[edge.to] ← tentative_gScore
                fScore ← tentative_gScore + H(edge.to, goal)
                pq.insert({fScore, edge.to})
            end if
        end for
    end while
end function
```

4.2.1 Step-by-Step Demonstration: A*

1. **Step 1 (Heuristic Initialization):** The visualizer displays the target node. Each node calculates and shows its $h(n)$ value (straight-line distance to the target).

2. **Step 2 (Guided Search):** Unlike Dijkstra's circular expansion, A* extracts nodes based on the lowest $f(n) = g(n) + h(n)$. The visualizer shows the search "pulling" preferentially towards the goal node.

3. **Step 3 (Path Discovery):** The algorithm terminates the moment the goal node is extracted from the priority queue, avoiding unnecessary exploration of the opposite side of the graph.

4.3 Johnson's Algorithm

Historical Context: Published by Donald B. Johnson in 1977, this algorithm was developed to efficiently solve the All-Pairs Shortest Path (APSP) problem on sparse graphs. While Floyd-Warshall is simple, its $O(V^3)$ complexity makes it prohibitively slow for graphs with few edges.

Johnson ingeniously combined Bellman-Ford and Dijkstra's algorithms, utilizing a mathematical technique to eliminate negative weights without altering the shortest paths.

For graphs with negative edges, we cannot simply use Dijkstra. We also cannot arbitrarily add a constant to all edges, as that disproportionately penalizes paths with more edges. Johnson's Algorithm solves this via a technique called **Edge Reweighting**.

4.3.1 How Johnson's Algorithm works

1. **Add a Dummy Node:** Insert a new virtual node S into the graph with 0-weight directed edges connecting to all existing nodes.

2. **Calculate Potentials:** Run Bellman-Ford starting from S to find the minimum distance $h(v)$ to each node. This step also detects if the graph has a negative cycle.

3. **Reweight Edges:** Adjust the weight of every original edge (u, v) using the calculated potentials:

$$w'(u, v) = w(u, v) + h(u) - h(v)$$

   This guarantees that all $w'(u, v) \geq 0$.

4. **Run Dijkstra:** With all non-negative edges, execute Dijkstra's algorithm from every vertex to find the shortest paths, then reverse the reweighting formula to get the true distances.

Johnson's Pseudocode:

Algorithm 7 Johnson's Algorithm

```
function JOHNSON(graph)
    G' ← graph with a new dummy vertex S
    for v ← 0 to V − 1 do
        G'.addEdge(S, v, 0)
    end for
    h ← BELLMANFORD(G', S)                          ▷ Calculates potentials h(v)
    if h indicates a negative cycle then
        return "Negative cycle detected"
    end if
    rew_graph ← empty graph
    for edge in graph.edges do
        w' ← edge.cost + h[edge.from] − h[edge.to]
        rew_graph.addEdge(edge.from, edge.to, w')
    end for
    dist ← 2D array of size V × V
    for u ← 0 to V − 1 do
        dijkstra_dist ← DIJKSTRA(rew_graph, u)
        for v ← 0 to V − 1 do
            dist[u][v] ← dijkstra_dist[v] − h[u] + h[v]     ▷ Restore true distances
        end for
    end for
    return dist
end function
```

Complexity Analysis:

- **Time Complexity: $\mathbf{O(V \cdot E \log V)}$**. We run Bellman-Ford once $O(V \cdot E)$ and Dijkstra V times. Using a binary heap, $V \times O((V + E) \log V) = O(VE \log V)$ for sparse graphs.
- **Space Complexity: $\mathbf{O(V^2 + E)}$**. We need $O(V + E)$ for the modified graph and $O(V^2)$ to store the final all-pairs distance matrix.

4.3.2 Step-by-Step Demonstration: Johnson's Algorithm

1. **Step 1 (Dummy Node Setup):** A new node S appears on the screen, instantly projecting 0-weight edges to every other node in the graph.
2. **Step 2 (Potential Calculation):** Bellman-Ford runs in the background, updating each node's $h(v)$ value.

3. **Step 3 (Edge Reweighting):** The visualizer iterates through all edges. The edge values visually shift from negative/mixed values to strictly non-negative values using the w' formula.

4. **Step 4 (Dijkstra Sweep):** Dijkstra's algorithm rapidly executes sequentially from node 1, then node 2, etc., populating the final APSP matrix efficiently.

5 Multi-Source Shortest Path (MSSP)

Sometimes, routing involves multiple starting locations simultaneously. A prime example is the **Nearest Fire Station** problem: *Given a city network with K fire stations and multiple fire emergencies, dispatch the engine from the station with the absolute shortest travel time to the fire.*

5.1 Super Source Simulation

Core Idea: Instead of running a Single-Source Shortest Path (SSSP) algorithm separately for every fire station (K times), which is heavily inefficient, we can reduce the problem mathematically by introducing a **"Super Source"** node.

1. Create a virtual **Super Source** node (S).

2. Add directed edges with 0 weight from S to all K fire station nodes.

3. Run standard Dijkstra's algorithm just **once** starting from S.

This traversal naturally expands outward from all fire stations simultaneously. By maintaining an `origin` tracking array, the algorithm can easily trace back and identify the exact originating fire station for any intersection.

MSSP Pseudocode:

Algorithm 8 MSSP - Super Source Dijkstra

```
function MSSP(graph, sources)
    S ← new virtual node (V)
    for src in sources do
        graph.addEdge(S, src, 0)
    end for
    dist ← [∞, ..., ∞]
    origin ← [−1, ..., −1]                ▷ Tracks which source reached this node first
    dist[S] ← 0
    pq ← empty priority queue (min-heap)
    pq.insert({0, S})
    while pq is not empty do
        u, d ← pq.poll()
        if d > dist[u] then continue
        end if
        for edge in graph[u] do
            v ← edge.to
            weight ← edge.cost
            if dist[u] + weight < dist[v] then
                dist[v] ← dist[u] + weight
                if u == S then
                    origin[v] ← v                 ▷ The source node itself
                else
                    origin[v] ← origin[u]         ▷ Inherit the origin from parent
                end if
                pq.insert({dist[v], v})
            end if
        end for
    end while
    return dist, origin
end function
```

Complexity Analysis:

- **Time Complexity:** $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$. We are performing exactly one standard Dijkstra run on a graph with $V+1$ nodes and $E+K$ edges.
- **Space Complexity:** $\mathbf{O}(\mathbf{V}+\mathbf{E})$. Required for the graph structures, priority queue, and the `dist`/`origin` tracking arrays.

5.1.1 Step-by-Step Demonstration: MSSP

1. **Step 1 (Super Source Creation):** A massive virtual node appears, sending 0-weight edges to all designated source nodes (e.g., the Fire Stations).

2. **Step 2 (Simultaneous Expansion):** The priority queue is initialized with the Super Source. As the algorithm runs, the visualizer shows a "wave" of shortest path expansions radiating out from *all* source nodes at the exact same time.

3. **Step 3 (Territory Claiming):** When a normal node is visited and its distance is finalized, it is color-coded to match the specific Fire Station that reached it first. Over time, this draws Voronoi-like routing zones across the graph.

6 Implementation Notes & Complexity

6.1 Implementation Notes

Our implementation is organized around a common graph interface (`IGraphStructure`) and multiple concrete graph representations.

- **Graph abstraction:** `IGraphStructure` provides a unified API so all shortest path methods can run on either adjacency list or adjacency matrix.

- **Heap implementation update:** Dijkstra-related methods now rely on dedicated heap headers in `SourceCode/Heap/`, including `IndexedPriorityQueue` (eager Dijkstra) and `IndexedDaryHeap` (D-ary optimized Dijkstra).

- **Primary runtime choice:** Benchmarking is executed on `AdjacencyList` because the provided datasets are sparse to medium-density and edge traversal dominates runtime.

- **Two-phase loading for large files:** The loader first scans the file to estimate vertex range and metadata, then performs a second pass to build the graph. This avoids repeated graph resizing during insertion.

- **Input robustness:** Each edge line can be either `u v` or `u v w`. Missing weights default to 1.

- **Practical runtime controls:** Floyd-Warshall is run on a bounded induced subgraph (first k vertices), and heavy algorithms can be skipped via command flags for large-scale datasets.

6.2 Complexity Notes in Practice

Theoretical complexities are summarized in Section 2. In implementation, the observed behavior is consistent with those bounds:

- **Dijkstra variants:** Fast on non-negative sparse graphs due to heap-based relaxations (binary heap, indexed priority queue, and indexed D-ary heap).
- **Bellman-Ford:** Significantly heavier for large edge sets because each full relaxation pass touches all edges.
- **Floyd-Warshall:** Cubic in vertices, therefore limited to a bounded subset for benchmark practicality.

6.3 Engineering Trade-offs

- **Adjacency List vs Matrix:** Adjacency list reduces memory for sparse graphs and is preferred for SSSP workloads; adjacency matrix is simpler for APSP semantics but grows as $O(V^2)$ in memory.
- **Benchmark fidelity vs runtime budget:** We keep full algorithm coverage, but expose skip options so seminar demonstrations remain responsive.

7 Applications

8 Quiz

9 Programming Assignments

10 Source Codes & Instructions

The source files are in `SourceCode/ShortestPath/`, while heap headers are in `SourceCode/Heap/`.

10.1 Dependencies

The benchmark now supports an optional Boost-backed heap backend. On macOS, install it with:

```
brew install boost
```

If Boost headers are unavailable, the benchmark automatically falls back to the custom heap implementations.

10.2 What Is Included

- Core graph structures: `AdjacencyList.*`, `AdjacencyMatrix.*`, `IGraphStructure.h`, `Edge.h`.
- Shortest path implementations: `ShortestPath.*`.
- Benchmark runner source: `read_graph_info.cpp`.
- Graph generator source: `graph_generator.cpp`.
- DAG graph generator source: `dag_graph_generator.cpp`.
- Heap implementations: `IndexedPriorityQueue.h`, `IndexedDaryHeap.h`, `BinaryHeap.h`.
- Datasets: `dataset1_sparse_large.txt`, `dataset2_dense_medium.txt`, `dataset3_complete_apsp.txt` `dataset4_negative_sssp.txt`, `dataset5_dag_sssp.txt`.
- Benchmark logs: `bench_dataset1_sparse_large_boost.log` to `bench_dataset5_dag_sssp_boost.log`

10.3 Input Graph Format

Each edge line is:

```
from to weight
```

If `weight` is omitted, the loader defaults to weight 1.

10.4 Compile Instructions (Maximum Optimization)

All commands below are compile-only and use aggressive optimization settings.

Working directory (from repository root):

```
cd SourceCode/ShortestPath
```

10.4.1 macOS (clang++)

Compile graph generator:

```
clang++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG graph_generator.cpp -o graph_generator
```

Compile DAG graph generator:

```
clang++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG dag_graph_generator.cpp -o dag_graph_generator
```

Compile benchmark runner:

```
clang++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG -I/opt/homebrew/include read_graph_info.cpp ShortestPath.cpp \
  AdjacencyList.cpp AdjacencyMatrix.cpp -o benchmark
```

10.4.2 Windows (MSYS2/MinGW-w64 g++)

Compile graph generator:

```
g++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG graph_generator.cpp -o graph_generator.exe
```

Compile DAG graph generator:

```
g++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG dag_graph_generator.cpp -o dag_graph_generator.exe
```

Compile benchmark runner:

```
g++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG read_graph_info.cpp ShortestPath.cpp \
  AdjacencyList.cpp AdjacencyMatrix.cpp -o benchmark.exe
```

10.4.3 Windows (MSVC cl)

Compile graph generator:

```
cl /std:c++20 /O2 /GL /DNDEBUG graph_generator.cpp \
  /link /LTCG /OUT:graph_generator.exe
```

Compile DAG graph generator:

```
cl /std:c++20 /O2 /GL /DNDEBUG dag_graph_generator.cpp \
  /link /LTCG /OUT:dag_graph_generator.exe
```

Compile benchmark runner:

```
cl /std:c++20 /O2 /GL /DNDEBUG read_graph_info.cpp ShortestPath.cpp \
  AdjacencyList.cpp AdjacencyMatrix.cpp /link /LTCG /OUT:benchmark.exe
```

Note: MSVC does not support `-march=native`. The closest equivalent is `/O2` with link-time optimization (`/GL + /LTCG`).

10.5 Run Workflow (After Compilation)

1. Generate a sample graph:

```
# macOS / Linux
./graph_generator 10000 50000 1 20 1 demo_graph.txt


# Windows (MinGW or MSVC shell)
graph_generator.exe 10000 50000 1 20 1 demo_graph.txt
```

Arguments:

- `10000`: number of vertices.
- `50000`: number of edges.
- `1 20`: minimum and maximum edge weights.
- `1`: directed graph (use `0` for undirected).

- `demo_graph.txt`: output filename.

2. Run benchmark:

```
# macOS / Linux
./benchmark dataset1_sparse_large.txt 3 200 --heap-backend=boost


# Windows (MinGW or MSVC shell)
benchmark.exe dataset1_sparse_large.txt 3 200 --heap-backend=boost
```

Benchmark positional arguments:

- `dataset1_sparse_large.txt`: input graph file path.
- `3`: number of source vertices used for SSSP runs.
- `200`: Floyd-Warshall vertex cap (induced subgraph size).

Benchmark flags (detailed):

- `-skip-bellman`: skips Bellman-Ford only. Use this for large graphs when negative-edge support is not being tested.
- `-skip-floyd`: skips Floyd-Warshall only. Use this when APSP is too expensive for the selected graph size.
- `-skip-heavy`: skips both Bellman-Ford and Floyd-Warshall. Recommended for quick demo runs focused on Dijkstra variants.
- `-heap-backend=custom`: uses the custom indexed heap implementations.
- `-heap-backend=boost`: uses Boost heap containers when Boost headers are installed.

Example full run (no skipping, small graph):

```
./benchmark dataset4_negative_sssp.txt 3 200
```

DAG benchmark note: for DAG inputs, the benchmark reports both the original `dagShortestPath` implementation and the fairer `dagShortestPath (precomputed topo order)` variant, which reuses one topological order across source runs.

10.6 Generate the Benchmark Datasets

```
./graph_generator 100000 500000 1 10000 1 dataset1_sparse_large.txt
./graph_generator 5000 10000000 1 100000 1 dataset2_dense_medium.txt
./graph_generator 1000 999000 1 1000 1 dataset3_complete_apsp.txt
./graph_generator 1000 25000 -100 1000 1 dataset4_negative_sssp.txt
```

10.7 Generate the DAG Benchmark Dataset

```
./dag_graph_generator 100000 500000 1 10000 1 dataset5_dag_sssp.txt
```

11 Datasets

We generated five directed weighted benchmark datasets to stress different shortest path regimes.

11.1 Dataset Suite

Test Dataset	**Vertices** (V)	**Edges** (E)	**Min W** (m)	**Max W** (M)	**Primary Algorithms t**
Sparse Large	100,000	500,000	1	10,000	Lazy vs. Eager vs. D-ary
Dense Medium	5,000	10,000,000	1	100,000	Eager vs. D-ary vs. Bellm
Complete (APSP)	1,000	999,000	1	1,000	Floyd-Warshall vs. ($V\times$
Negative SSSP	1,000	25,000	-100	1,000	Bellman-Ford vs. Floyd-W
Dag SSSP	100,000	500,000	1	10,000	DAG shortest path vs. D

11.2 Dataset Files

- `dataset1_sparse_large.txt`
- `dataset2_dense_medium.txt`
- `dataset3_complete_apsp.txt`
- `dataset4_negative_sssp.txt`
- `dataset5_dag_sssp.txt`

11.3 Header Format

Each dataset is emitted with metadata header lines, where the dataset name is taken from the output filename:

```
# Directed <dataset_name> graph dataset
# Nodes: <V> Edges: <E>
# FromNodeId    ToNodeId
```

followed by weighted edges in the form:

```
from to weight
```

11.4 Benchmark Interpretation

The DAG dataset is important because it highlights two different costs:

- `dagShortestPath`: recomputes the topological traversal each run.
- `dagShortestPath (precomputed topo order)`: reuses one topological order across source runs and is the fairer textbook-style comparison.

12 Comparison

References

[1] Dan Bernstein. Hash functions (djb2) - comp.lang.c usenet, 1991. Accessed: 2025-12-28.

[2] Eraser.io. Eraser diagram tool. https://eraser.io/, 2025. Online tool used to design UML diagrams and system architecture illustrations.

[3] Google. Gemini. https://gemini.google.com/, 2025. Generative AI model used to support progress tracking, multithreading concepts, cross-platform implementation, code refactoring, and report writing.

[4] OpenAI. Chatgpt. https://chat.openai.com/, 2025. AI assistant used to help draft technical reports, convert content to LaTeX, and format tables, diagrams, and documentation based on student-provided input.